



طراحی و پیاده سازی سامانه چندکاربره چت و اشتراک گذاری فایل با استفاده از Socket Programming و Shadow Control Channel

1- عنوان پروژه

در این پروژه، باید یک سامانه چندکاربره مبتنی بر معماری Client/Server طراحی و پیاده سازی کنید که در آن چند کاربر بتوانند به صورت همزمان به یک سرور مرکزی متصل شوند، در سیستم ثبت نام و ورود انجام دهند، با کاربران آنلاین چت کنند، فایل روی سرور آپلود کنند و فایل های آپلود شده را با کاربران دیگر به اشتراک بگذارند.

نکته مهم این پروژه، پیاده سازی یک کانال کنترلی از نوع Shadow Control Channel است. منظور از Shadow Control Channel این نیست که برای پیام های کنترلی یک Socket جداگانه ساخته شود. بلکه پیام های کنترلی باید روی همان اتصال اصلی داده ای ارسال شوند، اما به شکلی طراحی شوند که از داده های اصلی مانند پیام چت و محتوای فایل قابل تشخیص باشند.

2- هدف آموزشی پروژه

هدف این پروژه فقط ساخت یک برنامه ساده برای چت یا ارسال فایل نیست. هدف اصلی این است که مفاهیم مهم درس شبکه های کامپیوتری را در قالب یک پروژه عملی تجربه کنید در پایان پروژه، باید بتوانید مفاهیم زیر را به صورت عملی توضیح داده و پیاده سازی کند:

- ایجاد ارتباط بین Server و Client
- ارسال و دریافت داده از طریق شبکه
- مدیریت چند Client همزمان
- طراحی یک پروتکل ساده برای ارتباط بین Server و Client
- تفکیک منطقی پیام های کنترلی از داده های اصلی
- پیاده سازی Shadow Control Channel روی همان Socket داده
- ارسال پیام چت بین کاربران آنلاین
- آپلود فایل روی سرور
- اشتراک گذاری فایل آپلود شده با کاربران دیگر
- ارسال فایل به صورت Chunk
- مدیریت خطاهای رایج شبکه
- مدیریت قطع اتصال Client ها

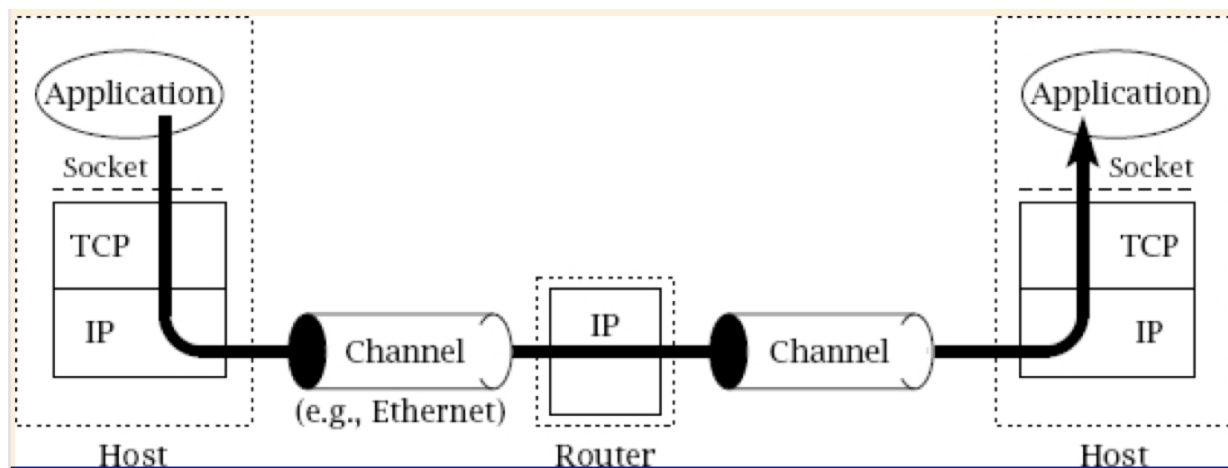
3- ایده کلی سیستم

در این پروژه، یک سرور مرکزی وجود دارد و تمام کاربران از طریق برنامه Client به این سرور متصل می‌شوند. سرور مسئول مدیریت کاربران، احراز هویت، نگهداری وضعیت کاربران آنلاین، مدیریت فایل‌های آپلودشده، دریافت پیام‌ها و هماهنگی انتقال فایل بین کاربران است.

برای مثال، فرض کنید دو کاربر با نام‌های arvin و moein در سیستم وجود دارند.

ابتدا کاربر arvin در سیستم ثبت‌نام می‌کند و وارد می‌شود. سپس یک فایل مانند report.pdf را روی سرور آپلود می‌کند. بعد از آن، کاربر moein نیز وارد سیستم می‌شود و فهرست کاربران آنلاین را مشاهده می‌کند. حالا کاربر arvin می‌تواند برای moein پیام چت ارسال کند یا فایل آپلودشده را با او به اشتراک بگذارد.

در این سیستم، سرور نقش هماهنگ‌کننده اصلی را دارد. یعنی Client‌ها برای انجام عملیات مهم، درخواست خود را به سرور



شکل ۱ - نمای کلی ارتباط برنامه‌ها از طریق Socket و TCP/IP

ارسال می‌کنند و سرور بر اساس وضعیت سیستم، پاسخ مناسب را برمی‌گرداند یا پیام و فایل را به کاربر مقصد منتقل می‌کند.

این شکل نشان می‌دهد که برنامه‌های کاربردی در دو سمت ارتباط، داده‌های خود را از طریق Socket به لایه TCP/IP تحویل می‌دهند و ارتباط واقعی از طریق شبکه برقرار می‌شود. در این پروژه نیز هر Client از طریق یک اتصال TCP با Server ارتباط برقرار می‌کند و تمام پیام‌ها، شامل پیام‌های کنترلی و داده‌های اصلی، در همین مسیر منتقل می‌شوند.

4- مفهوم Shadow Control Channel در این پروژه

در این پروژه، ارتباط بین هر Client و Server تنها از طریق یک Socket اصلی انجام می‌شود.

نباید برای ارسال پیام‌های کنترلی یک Socket جداگانه ایجاد کنید.

منظور از Shadow Control Channel این است که پیام‌های کنترلی مانند ثبت‌نام، ورود، خروج، مشاهده کاربران آنلاین و درخواست اشتراک‌گذاری فایل، روی همان Socket اصلی ارسال شوند، اما قالب آن‌ها به شکلی باشد که برنامه بتواند تشخیص دهد این پیام‌ها کنترلی هستند و نباید با داده‌های اصلی مانند محتوای فایل اشتباه گرفته شوند.

بنابراین در این پروژه، دو کانال فیزیکی یا دو Socket جداگانه نداریم. بلکه فقط یک اتصال TCP بین Client و Server وجود دارد و داخل همین اتصال، دو نوع داده به‌صورت منطقی از هم تفکیک می‌شوند:

1. پیام‌های کنترلی

2. داده‌های اصلی

پیام‌های کنترلی شامل دستورهایی مانند موارد زیر هستند:

REGISTER

LOGIN

LOGOUT

UPLOAD

LIST_FILES

LIST_USERS

SHARE

EXIT

داده‌های اصلی شامل موارد زیر هستند:

CHAT message

FILE chunk bytes

روش دقیق مخفی‌سازی یا تعبیه پیام‌های کنترلی در جریان داده بر عهده خودتان است و می‌تواند از روش‌هایی مانند header، frame structure، prefix، type field یا هر روش مناسب دیگر استفاده کنید؛ اما شرط اصلی این است که برای کنترل، Socket جداگانه ساخته نشود.

5- معماری کلی سیستم

سیستم از اجزای اصلی زیر تشکیل می‌شود:

1. Server

2. Clientها

3. بخش مدیریت کاربران

4. بخش مدیریت کاربران آنلاین

5. بخش مدیریت فایل‌ها

6. بخش مدیریت پیام‌های چت

7. بخش مدیریت اشتراک‌گذاری فایل

8. Shadow Control Channel روی همان Socket اصلی

9. بخش مدیریت خطاها

سرور باید بتواند چند Client را به صورت همزمان مدیریت کند. برای این کار می‌توان از Thread یا روش‌های مشابه استفاده کرد. اگر یک کاربر در حال ارسال فایل باشد، ارتباط کاربران دیگر نباید متوقف شود.

6- وظایف Server

سرور بخش مرکزی سیستم است و باید وظایف زیر را انجام دهد:

- دریافت اتصال از چند Client
- مدیریت همزمان چند کاربر
- ثبت نام کاربران جدید
- احراز هویت کاربران هنگام ورود
- نگهداری وضعیت کاربران آنلاین
- تغییر وضعیت کاربر به آفلاین هنگام خروج یا قطع اتصال
- دریافت و ذخیره فایل های آپلود شده
- ارسال فهرست فایل های آپلود شده
- ارسال فهرست کاربران آنلاین
- دریافت پیام چت و ارسال آن به کاربر مقصد
- دریافت درخواست اشتراک گذاری فایل
- اطلاع رسانی درخواست اشتراک گذاری به کاربر مقصد
- مدیریت پذیرش یا رد فایل توسط گیرنده
- ارسال فایل به صورت Chunk
- مدیریت خطاهای رایج مانند رمز اشتباه، کاربر آفلاین، فایل ناموجود و قطع اتصال

7- وظایف Client

Client برنامه ای است که توسط کاربر اجرا می شود. هر کاربر از طریق Client به سرور متصل می شود و دستورات خود را ارسال می کند.

وظایف Client عبارت اند از:

- اتصال به سرور
- ارسال درخواست ثبت نام
- ارسال درخواست ورود
- ارسال درخواست خروج
- دریافت دستور از کاربر
- ارسال پیام چت
- دریافت پیام چت
- ارسال درخواست آپلود فایل
- ارسال فایل به صورت Chunk
- مشاهده فهرست کاربران آنلاین

- مشاهده فهرست فایل‌های آپلودشده
- ارسال درخواست اشتراک‌گذاری فایل
- قبول یا رد درخواست دریافت فایل
- دریافت فایل و ذخیره آن
- نمایش پیام‌ها و پاسخ‌های سرور

Client باید بتواند همزمان هم ورودی کاربر را دریافت کند و هم پیام‌هایی را که از طرف سرور می‌رسند نمایش دهد. برای این کار می‌توان از Thread یا روش مناسب دیگر استفاده کرد.

8- پروتکل پیام‌ها

در این پروژه، ارتباط بین Client و Server باید بر اساس یک پروتکل مشخص انجام شود. باید قالب پیام‌ها را به‌گونه‌ای طراحی کنید که سرور و Client بتوانند نوع پیام را تشخیص دهند.

از آنجا که پیام‌های کنترلی و داده‌های اصلی باید روی یک Socket مشترک ارسال شوند، لازم است هر پیام دارای نوع مشخص باشد.

یک قالب پیشنهادی برای پیام‌ها می‌تواند به شکل زیر باشد:

TYPE	COMMAND	ARGUMENTS
------	---------	-----------

برای مثال، پیام‌های کنترلی می‌توانند با CTRL شروع شوند:

CTRL	LOGIN	arvin 1234
CTRL	LIST_USERS	
CTRL	SHARE	report.pdf moein

پیام‌های داده‌ای می‌توانند با DATA شروع شوند:

DATA	CHAT	moein سلام
DATA	FILE_CHUNK	report.pdf 1 1024 <bytes>
DATA	END_FILE	report.pdf

این قالب فقط یک پیشنهاد است. می‌توانید قالب دیگری طراحی کنید، اما باید در گزارش پروژه خود توضیح دهید که چگونه پیام‌های کنترلی را از داده‌های اصلی تفکیک کرده اید.

نکته بسیار مهم: حتی اگر در متن پروژه از عبارت کانال کنترلی و کانال داده استفاده می‌شود، منظور دو Socket جداگانه نیست. منظور فقط تفکیک منطقی پیام‌ها روی یک Socket واحد است.

9- دستورات کنترلی اصلی

دستورات کنترلی اصلی پروژه در ادامه آمده‌اند. این دستورات باید از طریق Shadow Control Channel و روی همان Socket اصلی ارسال شوند.

10-1 (دستور REGISTER)

کاربرد: ثبت نام کاربر جدید

قالب پیشنهادی:

```
REGISTER <username> <password>
```

مثال:

```
REGISTER arvin 1234
```

رفتار مورد انتظار سرور:

سرور باید بررسی کند که نام کاربری قبلاً ثبت نشده باشد. اگر نام کاربری آزاد باشد، کاربر جدید ثبت می‌شود.

پاسخ موفق:

```
OK
```

اگر نام کاربری تکراری باشد:

```
ERROR username already exists
```

10-2 (دستور LOGIN)

کاربرد: ورود کاربر به سیستم

قالب پیشنهادی:

```
LOGIN <username> <password>
```

مثال:

```
LOGIN arvin 1234
```

رفتار مورد انتظار سرور:

سرور باید بررسی کند که کاربر وجود دارد، رمز عبور درست است و کاربر از قبل آنلاین نیست.

پاسخ موفق:

```
AUTH_SUCCESS
```

پاسخ ناموفق:

```
AUTH_FAILED
```

اگر کاربر قبلاً آنلاین باشد:

```
ERROR user already logged in
```

10-3 دستور LOGOUT

کاربرد: خروج کاربر از حساب کاربری

قالب پیشنهادی:

LOGOUT

رفتار مورد انتظار سرور:

سرور باید وضعیت کاربر را از آنلاین به آفلاین تغییر دهد.

پاسخ موفق:

OK

اگر کاربر هنوز وارد سیستم نشده باشد:

ERROR authentication required

10-4 دستور UPLOAD

کاربرد: اعلام شروع آپلود فایل

قالب پیشنهادی:

UPLOAD <filename> <filesize>

مثال:

UPLOAD report.pdf 204800

توضیح:

این دستور فقط شروع آپلود را اعلام می‌کند. محتوای واقعی فایل باید بعد از تأیید سرور و به صورت Chunk روی همان Socket ارسال شود.

رفتار مورد انتظار سرور:

سرور باید بررسی کند که کاربر وارد سیستم شده باشد، نام فایل معتبر باشد و اندازه فایل معتبر باشد.

پاسخ موفق:

OK

خطاهای احتمالی:

ERROR authentication required
ERROR invalid filename
ERROR invalid filesize

10-5 دستور LIST_FILES

کاربرد: دریافت فهرست فایل‌های آپلودشده
قالب پیشنهادی:

پاسخ نمونه:

```
LIST_FILES
```

```
FILE_LIST
```

اگر فایلی وجود نداشته باشد:

اگر کاربر وارد سیستم نشده باشد:

```
FILE_LIST empty
```

```
ERROR authentication required
```

10-6 دستور LIST_USERS

کاربرد: دریافت فهرست کاربران آنلاین
قالب پیشنهادی:

پاسخ نمونه:

```
LIST_USERS
```

```
USER_LIST arvin ,moein,amirali
```

اگر کاربر وارد سیستم نشده باشد:

```
ERROR authentication required
```

10-7 دستور SHARE

کاربرد: ارسال درخواست اشتراک‌گذاری یک فایل آپلودشده با یک کاربر دیگر
قالب پیشنهادی:

مثال:

```
SHARE <filename> <target_user>
```

```
SHARE report.pdf moein
```

رفتار مورد انتظار سرور:

سرور باید بررسی کند که:

- فرستنده وارد سیستم شده باشد.
- فایل موردنظر وجود داشته باشد.

- کاربر مقصد وجود داشته باشد.
- کاربر مقصد آنلاین باشد.
- درخواست معتبر باشد.

اگر درخواست معتبر باشد، سرور برای کاربر مقصد پیام درخواست اشتراک‌گذاری ارسال می‌کند:

```
SHARE_REQUEST arvin report.pdf
```

خطاهای احتمالی:

```
ERROR file not found
ERROR target user not found
ERROR target user is offline
ERROR authentication required
```

10-8 دستور EXIT

کاربرد: خروج کامل Client و قطع اتصال از سرور

قالب پیشنهادی:

```
EXIT
```

رفتار مورد انتظار سرور:

اگر کاربر وارد سیستم شده باشد، سرور ابتدا وضعیت او را آفلاین می‌کند و سپس اتصال را به صورت کنترل‌شده می‌بندد.

```
OK
```

پاسخ پیشنهادی:

تفاوت EXIT و LOGOUT:

دستور LOGOUT فقط کاربر را از حساب خود خارج می‌کند، اما برنامه Client می‌تواند همچنان باز بماند. دستور EXIT یعنی Client قصد دارد اتصال خود را با سرور قطع کند و از برنامه خارج شود.

10- پیام‌های داده‌ای اصلی

پیام‌های داده‌ای برای انتقال محتوای واقعی استفاده می‌شوند. این پیام‌ها نیز باید روی همان Socket اصلی ارسال شوند، اما نوع آن‌ها باید از پیام‌های کنترلی قابل تشخیص باشد.

11-1 پیام CHAT

کاربرد: ارسال پیام چت به یک کاربر آنلاین

قالب پیشنهادی:

```
CHAT <target_user> <message>
```

مثال:

```
CHAT moein سلام ، فایل گزارش را برایت می‌فرستم
```

رفتار مورد انتظار سرور:

سرور باید بررسی کند که فرستنده وارد سیستم شده باشد، کاربر مقصد وجود داشته باشد و آنلاین باشد. سپس پیام را به کاربر مقصد ارسال کند.

پیام دریافتی در سمت مقصد می‌تواند به این شکل باشد:

```
CHAT_FROM arvin سلام ، فایل گزارش را برایت می‌فرستم
```

خطاهای احتمالی:

```
ERROR target user is offline  
ERROR target user not found  
ERROR authentication required
```

11-2 پیام ACCEPT_FILE

کاربرد: قبول کردن درخواست دریافت فایل

قالب پیشنهادی:

```
ACCEPT_FILE <filename>
```

مثال:

```
ACCEPT_FILE report.pdf
```

پس از پذیرش فایل، سرور باید فرآیند ارسال فایل را شروع کند یا اجازه شروع انتقال را بدهد.

پاسخ پیشنهادی:

```
SHARE_ACCEPTED
```

REJECT_FILE پیام (11-3)

کاربرد: رد کردن درخواست دریافت فایل

قالب پیشنهادی:

```
REJECT_FILE <filename>
```

مثال:

```
REJECT_FILE report.pdf
```

اگر گیرنده فایل را رد کند، انتقال فایل نباید انجام شود.

پاسخ پیشنهادی:

```
SHARE_REJECTED
```

11-4 پیام DATA برای ارسال Chunk فایل

کاربرد: ارسال بخشی از فایل

فایل نباید به صورت یکجا ارسال شود. فایل باید به چند بخش کوچکتر تقسیم شود که هر بخش را Chunk می‌نامیم.

قالب پیشنهادی:

```
DATA <filename> <chunk_number> <chunk_size>  
<bytes>
```

مثال مفهومی:

```
DATA report.pdf 1 1024  
<1024 bytes>
```

توضیح:

مقدار <bytes> متن معمولی نیست؛ بلکه محتوای واقعی فایل است و ممکن است شامل داده باینری باشد. بنابراین باید فایل‌ها را به صورت باینری بخوانید و بنویسید، نه فقط به صورت رشته متنی.

END_FILE پیام (11-5)

کاربرد: اعلام پایان ارسال فایل

قالب پیشنهادی:

```
END_FILE <filename>
```

مثال:

END_FILE report.pdf

وقتی سرور یا گیرنده این پیام را دریافت می‌کند، متوجه می‌شود که تمام Chunk های فایل ارسال شده‌اند و فایل می‌تواند بسته شود.

پاسخ موفق:

OK file received

11- انتقال فایل به صورت Chunk

در این پروژه، انتقال فایل باید به صورت Chunk انجام شود. یعنی فایل به بخش‌های کوچک‌تر تقسیم شده و هر بخش جداگانه ارسال می‌شود.

برای مثال، اگر یک فایل 4096 بایتی داشته باشیم و اندازه هر Chunk برابر 1024 بایت باشد، فایل به 4 بخش تقسیم می‌شود:

Chunk 1: bytes 0 to 1023

Chunk 2: bytes 1024 to 2047

Chunk 3: bytes 2048 to 3071

Chunk 4: bytes 3072 to 4095

ارسال فایل می‌تواند به شکل زیر باشد:

DATA report.pdf 1 1024

<1024 bytes>

DATA report.pdf 2 1024

<1024 bytes>

DATA report.pdf 3 1024

<1024 bytes>

DATA report.pdf 4 1024

<1024 bytes>

END_FILE report.pdf

اندازه پیشنهادی Chunk می‌تواند یکی از مقادیر روبرو باشد: 512,1024,2048,4096 بایت

پیشنهاد مناسب برای شروع پروژه 1024 بایت می‌باشد

12- نحوه پیاده‌سازی Shadow Control Channel

برای پیاده‌سازی Shadow Control Channel، باید روی یک Socket مشترک، پیام‌های مختلف را از هم تفکیک کنید. یک روش پیشنهادی، استفاده از یک فیلد نوع پیام در ابتدای هر پیام است. برای مثال:

```
CTRL|LOGIN|arvin |1234
CTRL|LIST_USERS
CTRL|UPLOAD|report.pdf|204800
DATA|CHAT|moein|سلام
DATA|FILE_CHUNK|report.pdf|1|1024|<bytes>
DATA|END_FILE|report.pdf
```

در این روش، سرور وقتی پیامی دریافت می‌کند، ابتدا نوع پیام را بررسی می‌کند: اگر پیام با CTRL شروع شود، آن را به بخش پردازش پیام‌های کنترلی می‌فرستد. اگر پیام با DATA شروع شود، آن را به بخش پردازش داده اصلی می‌فرستد. این روش باعث می‌شود پیام‌های کنترلی روی همان Socket داده‌ای منتقل شوند، اما از نظر منطقی از داده‌های اصلی جدا باشند. شما مجاز هستید از روش دیگری برای مخفی‌سازی یا تفکیک پیام‌های کنترلی استفاده کنید، اما باید در گزارش پروژه توضیح دهید که روش تان چگونه کار می‌کند و چرا نیاز به Socket جداگانه ندارد.

13- نکات مهم در طراحی پروتکل

در طراحی پروتکل، باید به نکات زیر توجه کنید:

1. هر پیام باید قالب مشخص داشته باشد.
2. سرور باید بتواند نوع پیام را تشخیص دهد.
3. پیام‌های کنترلی نباید از طریق Socket جداگانه ارسال شوند.
4. پیام‌های مربوط به فایل باید امکان حمل داده باینری را داشته باشند.
5. دستورهای نامعتبر نباید باعث توقف سرور شوند.
6. پاسخ‌های سرور باید ساده و قابل پردازش باشند.
7. قبل از اجرای دستورهای حساس، وضعیت Login کاربر باید بررسی شود.
8. قطع اتصال یک Client نباید باعث توقف کل سرور شود.
9. فایل ناقص نباید به‌عنوان فایل کامل در نظر گرفته شود.
10. پایان انتقال فایل باید با پیام مشخصی مانند END_FILE اعلام شود.

14- قابلیت‌های الزامی پروژه

باید حداقل قابلیت‌های زیر را پیاده‌سازی کنید:

1. اجرای سرور و اتصال چند Client به آن
2. مدیریت چند Client به صورت همزمان
3. استفاده از Socket Programming
4. ثبت نام کاربران
5. ورود و احراز هویت کاربران
6. خروج کاربران
7. نگهداری وضعیت کاربران آنلاین
8. مشاهده فهرست کاربران آنلاین
9. آپلود فایل روی سرور
10. مشاهده فهرست فایل‌های آپلودشده
11. ارسال پیام چت بین کاربران آنلاین
12. ارسال درخواست اشتراک‌گذاری فایل
13. قبول یا رد درخواست دریافت فایل
14. انتقال فایل به صورت Chunk
15. مدیریت خطاهای رایج
16. پیاده‌سازی Shadow Control Channel روی همان Socket اصلی
17. عدم استفاده از Socket جداگانه برای پیام‌های کنترلی

15- مدیریت خطاها

برنامه باید خطاهای رایج را مدیریت کند. برخی از خطاهای مهم عبارت‌اند از:

- نام کاربری تکراری
- نام کاربری یا رمز عبور اشتباه
- تلاش برای ورود کاربری که قبلاً آنلاین است
- اجرای دستور بدون Login
- ارسال پیام به کاربر آفلاین
- ارسال پیام به کاربر ناموجود
- درخواست فایل ناموجود
- آپلود فایل با نام نامعتبر
- آپلود فایل با اندازه نامعتبر
- قطع اتصال هنگام انتقال فایل

- ناقص ماندن فایل
- دریافت Chunk نامعتبر
- درخواست اشتراک گذاری نامعتبر
- دستور ناشناخته

نمونه پاسخ های خطا:

```
ERROR username already exists
ERROR authentication required
ERROR target user is offline
ERROR target user not found
ERROR file not found
ERROR invalid command
ERROR invalid chunk
ERROR connection lost during transfer
ERROR invalid SHARE request
```

16- رفتار مورد انتظار سیستم

رفتار کلی سیستم باید به شکل زیر باشد:

1. ابتدا Server اجرا می شود و منتظر اتصال Client ها می ماند.
2. هر Client به سرور متصل می شود.
3. کاربر می تواند ثبت نام یا ورود انجام دهد.
4. پس از ورود موفق، کاربر آنلاین محسوب می شود.
5. کاربر می تواند فهرست کاربران آنلاین را مشاهده کند.
6. کاربر می تواند فهرست فایل های آپلود شده را مشاهده کند.
7. کاربر می تواند فایل خود را روی سرور آپلود کند.
8. فایل باید به صورت Chunk ارسال شود.
9. کاربر می تواند با کاربران آنلاین چت کند.
10. کاربر می تواند یک فایل آپلود شده را با کاربر دیگر به اشتراک بگذارد.
11. کاربر مقصد می تواند دریافت فایل را قبول یا رد کند.
12. اگر فایل پذیرفته شود، انتقال آن انجام می شود.
13. اگر فایل رد شود، انتقال انجام نمی شود.
14. اگر کاربر خارج شود یا اتصال قطع شود، سرور باید وضعیت او را آفلاین کند.
15. قطع اتصال یک کاربر نباید باعث توقف سرور شود.

17- سناریوی نمونه اجرای پروژه

در این بخش، یک سناریوی ساده از اجرای پروژه نمایش داده می‌شود.

ابتدا Client A با نام کاربری arvin ثبت‌نام می‌کند:

Client A -> REGISTER arvin 1234

Server -> OK

سپس Client A وارد سیستم می‌شود:

Client A -> LOGIN arvin 1234

Server -> AUTH_SUCCESS

Client A یک فایل برای آپلود معرفی می‌کند:

Client A -> UPLOAD report.pdf 204800

Server -> OK

سپس Client A فایل را به‌صورت Chunk ارسال می‌کند:

Client A -> DATA report.pdf 1 1024

Client A -> DATA report.pdf 2 1024

...

Client A -> END_FILE report.pdf

Server -> OK file uploaded

حالا Client B با نام کاربری moein وارد سیستم می‌شود:

Client B -> LOGIN moein 5678

Server -> AUTH_SUCCESS

Client B فهرست کاربران آنلاین را دریافت می‌کند:

Client B -> LIST_USERS

Server -> USER_LIST arvin ,moein

Client A برای Client B پیام چت ارسال می‌کند:

Client A -> CHAT moein می‌فرستم گزارش را برایت

Server -> OK

Client B receives -> CHAT_FROM arvin سلام، فایل گزارش را برایت می‌فرستم

Client A درخواست اشتراک‌گذاری فایل را ارسال می‌کند:

Client A -> SHARE report.pdf moein

Server -> SHARE_REQUEST arvin report.pdf

Client B درخواست را قبول می‌کند:

Client B -> ACCEPT_FILE report.pdf

Server -> SHARE_ACCEPTED

سپس انتقال فایل آغاز می‌شود:

Server -> DATA report.pdf 1 1024

Server -> DATA report.pdf 2 1024

...

Server -> END_FILE report.pdf

اگر Client B درخواست را رد کند:

Client B -> REJECT_FILE report.pdf

Server -> SHARE_REJECTED

در این حالت، انتقال فایل انجام نمی‌شود.

18- دستورات پیشنهادی تکمیلی

دستورات زیر الزامی نیستند، اما می‌توانند برای کامل‌تر شدن پروژه استفاده شوند.

19-1 (دستور PING

برای بررسی زنده بودن اتصال:

PING

پاسخ سرور:

PONG

19-2 (دستور HELP

برای نمایش دستورات قابل استفاده:

HELP

پاسخ نمونه:

OK commands: REGISTER, LOGIN, LOGOUT, UPLOAD, LIST_FILES, LIST_USERS, SHARE, CHAT, EXIT

STATUS دستور (19-3

برای مشاهده وضعیت فعلی کاربر:

STATUS

پاسخ نمونه:

OK logged_in username=arvin

یا:

OK not_logged_in

CANCEL_TRANSFER دستور (19-4

برای لغو انتقال فایل:

پاسخ نمونه:

OK

یا:

ERROR no active transfer found

19- نکته نهایی درباره Shadow Control Channel

مهم‌ترین بخش این پروژه، درک درست مفهوم Shadow Control Channel است. در این پروژه، Shadow Control Channel به معنی ایجاد یک Socket جداگانه برای کنترل نیست. بلکه پیام‌های کنترلی باید در همان جریان داده‌ای اصلی قرار بگیرند و به شکلی طراحی شوند که برنامه بتواند آن‌ها را از داده‌های واقعی تشخیص دهد. به بیان دیگر، Client و Server فقط یک اتصال TCP اصلی دارند. داخل همین اتصال، هم پیام‌های کنترلی ارسال می‌شوند و هم داده‌های اصلی. تفاوت این دو نوع پیام باید با طراحی مناسب پروتکل مشخص شود. برای مثال، این طراحی قابل قبول است:

```
CTRL|LOGIN|arvin |1234
DATA|CHAT|moein|سلام
DATA|FILE_CHUNK|report.pdf|1|1024|<bytes>
CTRL|LIST_USERS
```

اما این طراحی قابل قبول نیست:

یک Socket برای کنترل

یک Socket جداگانه برای داده

20- سایر نکات مهم

در انجام پروژه به نکات زیر توجه کنید:

1. Client و Server باید دو برنامه جداگانه باشند.
2. سرور باید چند Client را همزمان مدیریت کند.
3. برای پیام‌های کنترلی نباید Socket جداگانه ساخته شود.
4. پیام‌های کنترلی و داده‌ای باید از نظر قالب پیام قابل تشخیص باشند.
5. فایل‌ها باید به صورت باینری خوانده و نوشته شوند.
6. فایل باید به صورت Chunk ارسال شود.
7. قطع اتصال یک Client نباید باعث Crash شدن سرور شود.
8. پاسخ‌های سرور باید ساده، مشخص و قابل پردازش باشند.
9. هیچ‌کدام از قابلیت‌های الزامی پروژه نباید حذف شوند.
10. روش دقیق تفکیک پیام‌های کنترلی از داده‌ها باید در گزارش توضیح داده شود.

بخش‌های امتیازی پروژه

موارد زیر الزامی نیستند، اما در صورت پیاده‌سازی صحیح می‌توانند به‌عنوان امتیاز اضافه در نظر گرفته شوند:

1. نمایش درصد پیشرفت آپلود و دانلود فایل
2. رمزنگاری ساده پیام‌های چت
3. ذخیره تاریخچه چت
4. ارسال پیام آفلاین
5. محدودیت حجم فایل
6. ثبت Log سمت سرور
7. استفاده از ACK برای تأیید دریافت Chunk ها
8. امکان Resume کردن انتقال فایل ناقص
9. نمایش وضعیت کاربران در Client
10. طراحی رابط کاربری متنی مرتب‌تر برای Client

نمونه نمایش درصد پیشرفت:

```
Uploading report.pdf: 45%
```

نمونه Log سمت سرور:

```
[INFO] user arvin logged in  
[INFO] file report.pdf uploaded by arvin  
[ERROR] target user moein is offline
```

11. پیاده‌سازی پروژه با زبان C

در صورتی که بخش اصلی پروژه، شامل Client و Server، با زبان C و با استفاده مستقیم از Socket Programming پیاده‌سازی شود، برای پروژه امتیاز اضافه در نظر گرفته می‌شود. هدف از این امتیاز، تشویق به درگیری مستقیم‌تر با مفاهیم سطح پایین شبکه، مدیریت اتصال‌ها، ارسال و دریافت داده، مدیریت خطاها و کار با فایل در سطح سیستم‌عامل است.

استفاده از زبان C الزامی نیست، اما پیاده‌سازی صحیح و کامل پروژه با C می‌تواند به‌عنوان امتیاز مثبت محاسبه شود.

باید یک فایل زیپ به فرمت **PRJ1_StudentNumber.zip** حاوی موارد زیر را تحویل دهید:

1. کد کامل Server
2. کد کامل Client
3. فایل توضیحی کوتاه پروژه
4. چند نمونه خروجی از اجرای برنامه
5. توضیح نحوه اجرای Server و Client
6. توضیح معماری کلی سیستم
7. توضیح قالب پیام‌ها و پروتکل طراحی شده
8. توضیح نحوه پیاده‌سازی Shadow Control Channel
9. توضیح نحوه مدیریت چند Client همزمان
10. توضیح نحوه مدیریت کاربران و فایل‌ها
11. توضیح نحوه مدیریت خطاها
12. توضیح اینکه چرا برای پیام‌های کنترلی Socket جداگانه ساخته نشده است